

GraphBench Challenge

Réfutation automatique de conjectures en théorie des graphes

Master 1 MIAGE — Optimisation Combinatoire
Année 2025–2026

Raphaël Meimoun

8 mai 2026

Conjectures réfutées	100 / 100
Score total (somme des temps)	7.2
Temps moyen de réfutation	0.07 s
GitHub	github.com/maladoooo/graphbench

1. Introduction et objectif

Le GraphBench Challenge consiste à réfuter automatiquement 100 conjectures en théorie des graphes. Chaque conjecture affirme qu'une inégalité entre invariants est vraie pour tous les graphes d'une certaine classe (connexes, sans griffe, arbres). L'objectif est de trouver un contre-exemple — un graphe qui viole strictement cette inégalité — en moins de 60 secondes par conjecture.

Notre programme maximise le score de violation : $violation(G) = A(G) - f(B(G))$ pour les conjectures de type $A \leq f(B)$, et l'opposé pour $A \geq f(B)$. Un contre-exemple est trouvé dès que $violation(G) > 0$. Le score officiel est la somme des temps de réfutation (120 s si échec).

2. Heuristique simple (Partie 1)

2.1 Représentation et génération initiale

Les graphes sont représentés via NetworkX. La génération initiale produit une population de 5 graphes aléatoires (Erdős-Rényi ou Barabási-Albert) avec $n \in [5, 11]$ sommets, selon la classe demandée. La réparation garantit l'appartenance à la classe après chaque mutation.

2.2 Mutations locales

Sept types de mutations sont appliquées aléatoirement à chaque itération :

Mutation	Description
Ajout/suppression d'arête	Modification du degré de deux sommets
Ajout/suppression de sommet	Avec reconnexion aléatoire au graphe
Densification locale	Ajout d'arêtes dans le voisinage d'un sommet
Subdivision d'arête	Insertion d'un sommet intermédiaire
Ajout de feuille	Extension linéaire du graphe
Fusion de sommets	Réduction du nombre de nœuds

2.3 Boucle de recherche

La boucle de recherche principale suit le schéma ci-dessous. Elle intègre une liste tabu (500 signatures d'arêtes) pour éviter de revisiter les mêmes graphes, et un mécanisme de redémarrage diversifié en cas de stagnation (aucune amélioration après 1500 itérations) :

```
population = generer_population_initiale()
tant que temps < limite:
    G = selectionner_candidat(population) # 70% best, 30% aleatoire
    H = muter(G)
    H = reparer(H, classe) # garantit l'appartenance a la classe
    si H dans liste_tabu: continuer
    score = calculer_violation(H, conjecture)
    si score > meilleur_score: mettre_a_jour_best()
    si violation > 0: retourner(contre_exemple)
    mise_a_jour_population()
```

2.4 Réparation et filtrage

Après chaque mutation, le graphe est réparé : reconnexion si déconnecté (ajout d'arête entre composantes), suppression des griffes induites pour les graphes sans griffe. Pour les invariants lents (diamètre, rayon), les graphes de plus de 80 sommets sont ignorés.

2.5 Filtres intelligents (Smart Filters)

Avant la recherche locale, `generate_smart()` tente de construire directement un contre-exemple selon la structure de la conjecture. Cette optimisation clé a permis de résoudre 70+ conjectures en moins de 1 ms :

Type de conjecture	Stratégie	Justification
second_smallest_laplace_eigenvalue	Barbell $K_k - e - K_k$	$\lambda_k \rightarrow 0$ quand $k \rightarrow \infty$
remoteness ou proximity	Graphe dense (complet - quelques chemins courts)	Chemins courts = grand remoteness
diameter + claw_free	Cycle C_n	Grand diamètre sans griffe
matching + tree	Étoile ou chemin	Structures extrêmes pour μ
domination + general	Clique ou barbell	γ petit sur graphes denses

Cas critique — connectivité algébrique : La fonction `NetworkX nx.algebraic_connectivity()` utilise le solveur ARPACK, qui diverge numériquement sur les graphes barbell ($\lambda_k \approx 10^{-15}$, très proche de zéro). Cela provoquait des blocages de 20 secondes par conjecture. Le remplacement par `numpy.linalg.eigvalsh(laplacian_matrix)` — déterministe et $O(n^3)$ — a résolu toutes les conjectures impliquant λ_k en moins de 0.3 s.

3. Architecture FunSearch (Partie 2)

FunSearch [Romera-Paredes et al., 2023] est une architecture d'évolution guidée par LLM. L'idée est de ne plus se limiter à $score(G) = violation(G)$, mais de chercher automatiquement une fonction plus informative qui guide la recherche vers des graphes prometteurs avant même qu'ils soient des contre-exemples.

3.1 Cycle d'évolution

- Initialisation :** la fonction de base est *return violation*.
- Évaluation :** chaque fonction est testée sur un sous-ensemble de 10 conjectures difficiles pendant 5 s chacune. Le score est le nombre de conjectures réfutées + bonus pour les violations élevées.
- Proposition LLM :** les 3 meilleures fonctions sont envoyées à Claude avec le prompt : *"Proposer une variante améliorée de cette fonction de score heuristique"*.
- Filtrage et rétention :** les nouvelles fonctions valides (pas d'erreur Python, retour numérique) remplacent les moins bonnes dans le pool.
- Répétition :** le cycle tourne pendant 5 générations ($\approx$ 15 minutes).

3.2 Résultat : fonction heuristique retenue

Après évolution, la meilleure fonction (score = 10/10 conjectures de test réfutées) est :

```
def heuristic_score(G, invariants, conjecture):
    violation = conjecture.violation(invariants)
    n = invariants.get("order", G.number_of_nodes())
    m = invariants.get("size", G.number_of_edges())
    diam = invariants.get("diameter", 0)
    Delta = invariants.get("maximum_degree", 0)
    triangles = invariants.get("triangle_number", 0)
    density = 2*m / (n*(n-1)) if n > 1 else 0
```

```
return (10.0*violation + 0.3*diam + 0.2*Delta  
        + 0.1*triangles - 0.005*n - 0.1*abs(density - 0.5))
```

Le coefficient $-0.005n$ (au lieu de l'initial $-0.05n$) est crucial : une pénalité trop forte bloquait la recherche sur de petits graphes ($n \leq 6$), alors que certaines conjectures nécessitent des graphes de taille $n \geq 15$.

4. Résultats expérimentaux

4.1 Résultats globaux

Métrique	Valeur
Conjectures réfutées	100 / 100 (100 %)
Score total (somme des temps)	7.2 s
Temps moyen de réfutation	0.07 s
Temps maximum (conjecture #3368)	1.17 s
Temps minimum	< 1 ms
Conjectures résolues par smart filter	~75 / 100
Conjectures résolues par recherche locale	~25 / 100

4.2 Résultats par classe de graphes

Classe	Nb conjectures	Réfutées	Taux
connected (connexe)	44	44	100 %
claw-free + connected (sans griffe)	50	50	100 %
connected + tree (arbre)	6	6	100 %

4.3 Cas difficiles et solutions

Trois conjectures ont nécessité des développements spécifiques :

Conjecture	Invariants	Problème	Solution
#7711–#7743 (8 conjectures)	λ_2 (second_smallest_laplace_eigenvalue)	ARPACK diverge sur barbell ($\lambda_2 \approx 0$)	numpy.eigvalsh déterministe
#3368	remoteness vs matching_number	Graphe optimal $n = 15+$	Smart filter dense + multi-seed
#5998, #6013	radius vs independence_number	Seed 42 ne converge pas → seed 7 et 13	Recherche multi-graine

5. Discussion scientifique

5.1 Quelles conjectures sont faciles à réfuter ?

Les conjectures les plus faciles font intervenir des invariants calculables en temps constant (ordre, taille, degrés). Les 40+ conjectures impliquant *density*, *maximum_degree*, *triangle_number* ou *domination_number* sont toutes réfutées en moins de 50 ms, souvent dès le filtre intelligent initial. Les graphes complets K_n ou les graphes étoile S_n fournissent des contre-exemples immédiats pour les bornes simples sur les degrés.

5.2 Quels invariants sont difficiles à manipuler ?

Deux catégories d'invariants posent problème. D'abord, les **invariants de chemin global** (diamètre, rayon, proximité, remoteness) : ils nécessitent un BFS depuis chaque sommet en $O(n+m)$, rendant les grands graphes prohibitifs. Nous limitons la recherche à $n \leq 80$ pour ces invariants. Ensuite, la **connectivité algébrique** λ : le solveur ARPACK de NetworkX diverge sur les graphes barbell car $\lambda \approx 10^{-15}$. Ce bug silencieux a nécessité plusieurs heures de débogage et le remplacement par `numpy.eigvalsh`.

5.3 Quelles mutations sont efficaces ?

L'ajout et la suppression d'arêtes sont les mutations les plus utilisées ($\approx 60\%$ des itérations réussies). La densification locale est particulièrement efficace pour les conjectures sur le matching et la domination. Les mutations de subdivision créent des graphes de grande excentricité utiles pour les invariants de diamètre. La fusion de sommets aide à trouver des contre-exemples pour les bornes supérieures sur α (*independence_number*).

5.4 FunSearch améliore-t-il réellement l'heuristique ?

Oui, mais marginalement sur ce benchmark. La modification la plus impactante identifiée par FunSearch est la réduction du coefficient de pénalité de taille ($-0.05n \rightarrow -0.005n$). Sans cette correction, la recherche converge vers de petits graphes ($n \leq 6$) et rate les contre-exemples nécessitant $n \geq 15$. Les bonus sur le diamètre ($+0.3 \times \text{diam}$) aident les conjectures de diamètre mais sont neutres pour les autres. L'architecture FunSearch est surtout utile pour identifier ces équilibres sans test manuel exhaustif.

5.5 L'IA a-t-elle produit des idées utiles ?

L'IA (Claude) a contribué à plusieurs niveaux. Pour le **débogage**, la suggestion d'utiliser `numpy.eigvalsh` au lieu d'ARPACK a résolu 8 conjectures bloquées. Pour la **stratégie de recherche**, la recherche multi-graine avec allocation temporelle équitable est une idée issue du dialogue avec le LLM. Pour les **smart filters**, l'identification des graphes barbell comme cas canonique pour $\lambda \rightarrow 0$ et des graphes denses pour remoteness provient d'une analyse guidée par l'IA des propriétés mathématiques des invariants. En revanche, les tentatives de FunSearch pour ajouter des bonus invariant-spécifiques ont souvent dégradé les performances en créant des biais trop forts.

5.6 Limites et perspectives

Notre approche atteint 100/100 sur ce benchmark, mais repose sur des smart filters spécifiques aux types de conjectures présents. Une généralisation nécessiterait un système de détection automatique de la structure de la conjecture pour sélectionner le filtre approprié. Les techniques

d'apprentissage par renforcement (AlphaGraph) ou les méthodes de recuit simulé pourraient améliorer davantage les cas difficiles nécessitant une exploration guidée fine de l'espace de graphes.

Références : Romera-Paredes et al. (2023). Mathematical discoveries from program search with large language models. *Nature*. | Hagberg et al. (2008). Exploring network structure with NetworkX. *SciPy Conf*.